Fiche d'information

Référence	N°219 v1.1	Date	14.10.2013
Objet	**PEC+ version TIRAT**- Aide à l'implémentation du WSDL (ServicePECB2_EXP_4.0.4)		

Type	☒ Technique	☐ Pratique	☐ Communication	☒ Livraison
Destinataires	☒ Éditeurs	☐ Équipementiers	☐ OCT	☐ CIS
Produits	Facturation : ☐ CDC Editeurs ☐ CDC OCT ☐ FSV agrément ☐ FSV exploitation ☐ SCOR	Référentiels : ☐ Dispositif Intégré ☐ Terminal lecteur ☐ TLA ☐ Lecture Vitale ☐ Télé-mise à jour	Produits d'infrastructure : ☐ ATSAM ☐ SrvSVCNAM ☐ GALSS ☐ Cryptolib ☐ MICA Services en Ligne : ☐ CDR ☐ Espace pro	Télé-services intégrés : ☒ Tous Téléservices concernés par le cadre d'interopérabilité Acquisition des droits : ☐ API de Lecture
Secteur	☒ Libéral		☐ Hospitalier	
Professionnels de Santé concernés	☐ Médecins ☐ Sages-femmes ☐ Chirurgiens-dentistes ☐ Pharmaciens ☐ Laboratoires	☐ Auxiliaires médicaux ☐ Centres de Santé ☐ Fournisseurs ☒ Transporteurs	☐ Établissements publics ☐ Cliniques privées ☐ ESPIC	
Prise en compte	☒ Facultative	☐ Obligatoire		

***Nota** : les modifications apportées par rapport à la version précédente sont surlignées en jaune.*

Contexte

Le WSDL du service PEC+ est adapté pour vous permettre de l'utiliser directement sous JAVA et .NET dans la majorité des IDE du marché, sans que vous ayez à le modifier. Ce WSDL est testé pour .NET (Visual Studio) et pour JAVA (Eclipse).

- La valeur de l'attribut « transport » de l'élément `<wsdl :binding>` a été mise à jour à « http://schemas.xmlsoap.org/soap/http » au lieu de « http://www.w3.org/2003/05/soap/bindings/HTTP/ ».
- La ligne `<wsaw:UsingAddressing required= "true"/>` a été supprimée.

L'aide à l'implémentation décrite ci-dessous précise les impacts et les adaptations à effectuer dans le support de de la formation « TLSi - Supports de formation / Aide au développement » qui a été mis à votre disposition (https://diffusion.sesam-vitale.fr/dossiers/voir.asp?id=669).

Fiche d'information

Valorisation du header Action

Pour que le code généré valorise automatiquement le header Action avec la bonne valeur tel que spécifié dans le WSDL, il faut indiquer que le WS-Addressing doit être mis en œuvre :

En java

Lors de l'instanciation de la classe du service Port contenant les méthodes d'appel des opérations.
Pour ce faire, il faut récupérer auprès du service l'interface du service port en passant en paramètre une instance de classe correspondant à la gestion du WS-Addressing :
Exemple pour le TELESERVICE « AAT » - Avis d'Arrêt de Travail :

```
AATService service = aat.getAATService(new javax.xml.ws.soap.AddressingFeature());
```

En C# / .NET

Dans le fichier app.config généré lors de l'importation du WSDL, valoriser l'attribut "*messageVersion*" de l'élément *textMessageEncoding* du binding à "*Soap12WSAddressing10*".
Exemple pour le TELESERVICE « AAT » - Avis d'Arrêt de Travail :

```
<configuration>
    <system.serviceModel>
        <bindings>
            <customBinding>
                <binding name="AATServiceBinding">
                    <textMessageEncoding maxReadPoolSize="64" maxWritePoolSize="16"
                        messageVersion="Soap12WSAddressing10" writeEncoding="utf-8">
                        ...
                    </textMessageEncoding>
                    ...
                </binding>
            </customBinding>
        </bindings>
        ...
    </system.serviceModel>
</configuration>
```

URL du service

L'URL du service doit pouvoir être modifiée en fonction de l'environnement appelé (de tests ou de production, voir indications dans le guide d'intégration). Afin de faire cette modification facilement, il convient de paramétrer l'URL appelée.

Exemple pour le TELESERVICE « AAT » - Avis d'Arrêt de Travail :

En java

```
String URL_AAT = https://qualif.services-ps.ameli.fr/lps;
AAT aat = new AAT(url);
AATService service = aat.getAATService(new javax.xml.ws.soap.AddressingFeature());
((BindingProvider)service).getRequestContext().put(BindingProvider.ENDPOINT_ADDRESS
_PROPERTY,
                                                    URL_AAT);
```

En C# / .NET

```
String URL_AAT = "https://qualiflps.services-ps.ameli.fr/lps";
AATServiceClient aat = new AATServiceClient();
aat.Endpoint.Address = new EndpointAddress(URL_AAT);
```

Ainsi, il suffit d'un fichier de configuration de l'application dans lequel se trouve indiquées l'URL de test et l'URL d'exploitation du service sans avoir à modifier l'application.

Fiche d'information

Erreur SOAP

Le fichier WSDL ne fait pas référence aux erreurs SOAP de type Erreur (correspondant au détail de l'erreur SOAP défini dans le cadre d'interopérabilité). En effet, il n'est pas exclu de recevoir une erreur SOAP qui ne corresponde pas à ce type erreur. De ce fait, les exceptions levées ne seront que des SOAP exceptions (le type « Erreur » n'est pas implémenté).

~~En java~~,il convient donc :

- de coder de sorte à ne prendre en compte que des SOAPFaultException
- de parcourir l'arbre DOM de ce type d'objet, afin de lire les données nécessaires à la traçabilité des erreurs (notamment rechercher dans l'élément detail un élément Erreur tel que défini dans le cadre d'interopérabilité).

Exemple :

```
catch (SOAPFaultException e) {
  SOAPFault faute = e.getFault();
  log.debug("SOAPFaultException : Code = " + faute.getFaultCode()
          + " - Reason text = " + faute.getFaultString());

  Iterator subCodeList = faute.getFaultSubcodes();
  while (subCodeList.hasNext()){
    String subCodeText = ((QName)subCodeList.next()).toString();
    if (subCodeText != null){
      log.debug("                    - subCodeText = " + subCodeText);
    }
  }

  // recuperation de l'element Erreur dans le Detail de la faute soap
  if (faute.getDetail().hasChildNodes()) {
    Iterator listeDetail = faute.getDetail().getDetailEntries();
    while (listeDetail.hasNext()){
      DetailEntry d = (DetailEntry)listeDetail.next();
      log.debug("DetailEntry = " + d.getElementName().getLocalName());
      if (d.getElementName().getLocalName().equals("Erreur")){
        String code = d.getAttributeValue(new QName("code"));
        String messageID = d.getAttributeValue(new QName("messageID"));
        String severite = d.getAttributeValue(new QName("severite"));
        String value = d.getValue();
        log.debug("value = " + value);
        log.debug("code = " + code + ", messageID = " + messageID + ", severite = " + severite);
      }
    }
  }
}
```